

Cómo conectar HTML con Google Sheets sin morir en el intento de CORS

El patrón comprobado para que tus portales lean y escriban en Google Sheets vía Apps Script, sin el error de CORS que aparece y desaparece.

Reutilizable en todos los portales POC / EVA+ / clientes

La idea en una frase: en Apps Script **no puedes agregar headers de CORS**. Por eso la solución no es “agregar CORS” (eso es imposible ahí), sino **no provocar el preflight** que Apps Script no sabe responder. Si entiendes esto, el problema desaparece para siempre.

1. Por qué te pasa (y por qué “oscila” entre POST y GET)

Cuando el navegador hace una petición a otro dominio (tu HTML en GitHub Pages → Google), aplica CORS. Hay dos cosas que pueden romperse:

- **El preflight:** si tu petición “no es simple”, el navegador manda primero un `OPTIONS`. Apps Script no responde ese OPTIONS con los headers correctos → se bloquea **todo**. Esto es lo que ves como “falla de POST”.
- **Leer la respuesta:** Google *sí* devuelve `Access-Control-Allow-Origin: *` en la respuesta real del `/exec`, así que *sí* puedes leerla... siempre y cuando no te haya matado antes el preflight.

🚫 **El detonante #1 del preflight:** poner `Content-Type: application/json`. Ese header convierte tu POST en “no simple” → preflight → muerte. Cambiar a GET para escapar te quita el body y te lleva a otros problemas: por eso parece que “a veces es POST, a veces es GET”. **El enemigo es el Content-Type, no el método.**

2. La regla de oro: manda una “petición simple”

Una petición **no dispara preflight** si el `Content-Type` es uno de estos tres: `text/plain`, `application/x-www-form-urlencoded` o `multipart/form-data` (y no llevas headers personalizados). El truco es dejar que el body viaje así, y parsear el JSON tú mismo en el Apps Script.

3. Patrón recomendado — POST “simple” con URLSearchParams

Front-end · un solo helper para leer y escribir

```
// Reúsalo en TODOS tus portales. El secreto es que NO lleva headers.
const GS_URL = "https://script.google.com/macros/s/TU_ID/exec";

async function gs(action, data) {
  const res = await fetch(GS_URL, {
    method: 'POST',
    body: new URLSearchParams({ action, data: JSON.stringify(data) || {} })
    // SIN objeto headers. URLSearchParams manda x-www-form-urlencoded,
    // que es "simple" => NO hay preflight => no hay CORS que rompa.
  });
  return res.json();
}

// Uso:
// const pipeline = await gs('getPipeline', { vendedor: 'David' });
// await gs('saveProspecto', { nombre: 'Aceros SA', etapa: 'Lead' });
```

Back-end · plantilla de Apps Script que aguanta GET, POST y JSONP

```
function doGet(e) { return handle(e); }
function doPost(e) { return handle(e); }

function handle(e) {
  const p = (e && e.parameter) || {};
  let data = {};
  try { if (p.data) data = JSON.parse(p.data); } catch (_) {}
  // por si algún cliente manda el JSON crudo en el body:
  if (e && e.postData && e.postData.contents && !p.data) {
    try { data = JSON.parse(e.postData.contents); } catch (_) {}
  }
}
```

```

let out;
switch (p.action) {
  case 'getPipeline': out = getPipeline(data); break;
  case 'saveProspecto': out = saveProspecto(data); break;
  // ...tus demás acciones
  default: out = { ok: false, error: 'Accion no reconocida: ' + p.action };
}

const json = JSON.stringify(out);
// Si pidieron callback => respondemos JSONP; si no, JSON normal.
return p.callback
  ? ContentService.createTextOutput(p.callback + '(' + json + ')')
    .setMimeType(ContentService.MimeType.JAVASCRIPT)
  : ContentService.createTextOutput(json)
    .setMimeType(ContentService.MimeType.JSON);
}

```

✓ **Por qué funciona:** el body va como `x-www-form-urlencoded` (simple, sin preflight), llega a `e.parameter.data`, lo parseas con `JSON.parse`, y Google añade el header que deja leer la respuesta. Sirve igual para lecturas y escrituras.

4. Plan B blindado — JSONP (ignora CORS por completo)

Si en algún proyecto sigues peleando, usa JSONP: una etiqueta `<script>` no está sujeta a CORS. Solo sirve para GET y el dato viaja en la URL (mantenlo por debajo de ~unos KB).

```

function jsonpFetch(url) {
  return new Promise((resolve, reject) => {
    const cb = 'cb_' + Math.random().toString(36).slice(2);
    const s = document.createElement('script');
    window[cb] = d => { delete window[cb]; s.remove(); resolve(d); };
    s.onerror = () => reject(new Error('JSONP error'));
    s.src = url + '&callback=' + cb;
    document.body.appendChild(s);
  });
}

// Uso (el back de arriba ya soporta &callback):
// jsonpFetch(GS_URL + '?action=getPipeline&data=' + encodeURIComponent(JSON.stringify({})))

```

5. Lista negra — lo que NUNCA debes hacer

⚡ No hagas esto:

- `headers: { 'Content-Type': 'application/json' }` → dispara el preflight → muere.
- Intentar poner `Access-Control-Allow-Origin` en el Apps Script → **imposible** con `ContentService`, no pierdas tiempo.
- `mode: 'no-cors'` → la petición sale, pero la respuesta es *opaque*: no la puedes leer. Inútil para traer datos.
- Creer que cambiar GET ↔ POST arregla algo. El problema es el `Content-Type`.

6. Diagnóstico rápido — síntoma → causa → solución

SÍNTOMA EN CONSOLA	CAUSA REAL	SOLUCIÓN
"blocked by CORS policy / preflight"	Mandaste <code>Content-Type: application/json</code>	Quita el header; usa <code>URLSearchParams</code> (sección 3)
La petición sale pero <code>res.json()</code> da vacío/opaque	Usaste <code>mode: 'no-cors'</code>	Quítalo; con el patrón simple ya puedes leer
"Acción no reconocida: undefined"	El router está vivo pero no mandaste <code>action</code>	Es buena señal: el backend responde. Manda <code>action</code>
Lee datos viejos / "archivo no existe"	Usaste la URL <code>googleusercontent.com</code>	Usa siempre la URL <code>/exec</code> de <code>script.google.com</code>
Cambié el código y no se refleja	No redespelgaste versión nueva	Administrar implementaciones → Nueva versión

7. Checklist de despliegue del Apps Script

- 1 Implementar → **Nueva implementación** → tipo **Aplicación web**.
- 2 **Ejecutar como:** Yo (el dueño de la hoja).
- 3 **Quién tiene acceso:** Cualquier persona.

4 Copia la URL que termina en `/exec` (esa es la publica y permanente).

5 Al actualizar el código, redepliega como **Nueva versión** sobre la misma implementación para **conservar la URL**.

🔗 **Tip de depuración:** abre tu URL `/exec` directamente en el navegador (con `?action=getPipeline` , etc.). Si responde JSON, el backend está bien y el problema es de front-end. Si no, el problema es de despliegue o del Apps Script.